

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance (BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks:100

Annexure A

Debugging or Optimization task

Code of Conduct:

*I **Dhanya Sri M (192572099)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Question No.	Problem Identification (20%)	Debugging(25%)	Optimization (20%)	Analysis (20%)	Code Quality (15%)	Total Marks (100)
Q1						
Q2						
Q3						
Q4						

Q5						
Q6						
Q7						
Q8						
Q9						
Q10						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

21) Deadlock Debugging

Cause of Deadlock

1. P1 locks R1 and waits for R2.
2. P2 locks R2 and waits for R1.
3. Both Processes wait indefinitely, creating a circular wait

Deadlock Conditions

1. Mutual Exclusion
2. Hold and Wait
3. No Preemption
4. Circular Wait

All four conditions are present

Solution

Use resource ordering :

Both processes should acquire resource in the same order ($R1 \rightarrow R2$)

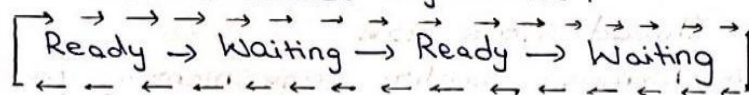
Conclusion

Deadlock occurs due to circular waiting. A fixed resource acquisition order prevents deadlock.

22) CPU Utilisation Problem

Cause

The scheduler log shows :



This indicates processes are frequently waiting for I/O operations.

Effects

1. CPU remains idle
2. Low CPU utilisation (30%.)
3. Reduced throughput

Optimisation:

1. Increase multiprogramming
2. Use asynchronous I/O
3. Improve scheduling algorithms
4. Use caching and buffering

Conclusion

Frequent I/O wait cause poor CPU utilisation. Better scheduling and I/O management improve performance.

Q3) Memory Bottleneck Debugging

Memory Usage

Code = 5MB

Heap = 150MB

Stack $\Rightarrow 25 \times 1\text{MB} = 25\text{MB}$

Total = 180MB

Cause

Frequent page fault occur, because:

1. Multiple threads access memory simultaneously.
2. Poor memory locality
3. Heap fragmentation

Optimisation

Reduce thread stack size

Improve memory locality using memory pools and contiguous allocation

Starvation Debugging

Cause

1. Process P5 in Q3 (lowest Priority)
2. Higher-priority processes in Q1 and Q2 continuously receive CPU time

Result

1. P5 waits indefinitely
2. This condition is called starvation.

Optimisation

Use Aging:

- Increase the priority of processes that wait for a long time

Benefits

1. Prevent starvation
2. Ensure
3. Increase Priority

Q5) Context - Switch Overhead Debugging

Given:

Quantum = 1ms

Burst time = 2ms

High % frequency

Cause: Threads are preempted before finishing, causing excessive context switches.

Optimisation

Increase time quantum (eg, 4-8ms)

Reduce % wait using asynchronous I/O

6) Producer - Consumer

Error: mutex is acquired before empty/full. A thread may block while holding the mutex

Wrong Order

wait(mutex)
wait(empty/full)

Correct Producer

wait(empty)
wait(mutex)
insert(item)
signal(mutex)
signal(full)

Consumer

wait(full)
wait(mutex)
remove(item)
signal(mutex)
signal(empty)

7) Thread Performance

Problem: One thread is created for every request

Effects:

High CPU overhead
Increasing memory usage
Many context switches

Optimisation

Use a thread pool
Use event-driven/asynchronous I/O

Result: Better scalability and stable performance

8) Banker's Algorithm

Need Matrix

Process	Need (A, B, C)
P1	(1, 1, 1)
P2	(1, 1, 1)
P3	(2, 1, 1)

Request by P2 = (1, 1, 1)

Request $\leq$ Need

Request $\leq$ Available

After allocation, a safe sequence exists

P2 $\rightarrow$ P1 $\rightarrow$ P3

Conclusion

The request can be granted safely

9) Deadlock Detection and Recovery

Wait-For Cycle

T1 $\rightarrow$ T2 $\rightarrow$ T3 $\rightarrow$ T4 $\rightarrow$ T1

Deadlock: Yes, a cycle exists

Detection: Construct a Wait-For Graph and check for cycles

Recovery Methods

Abort one transaction

Resource preemption and rollback

Best choice

Abort one victim transaction because it causes minimum disruption.

10) Monitor-Based Synchronization

Error: Waiting processes are never awakened because signal() is missing

Wrong Logic

if (busy) wait();

Correct Logic

while (busy)

wait(printerfree);


```
busy = true;  
// printing  
busy = false;  
signal (printFree);
```

Why:

while rechecks the condition and signal C)
wakes a waiting process

Result: Correct synchronization and fairness

Conclusion:

These answers cover

1. Deadlock
2. CPU Scheduling
3. Memory management
4. Starvation